

Procesamiento Básico de Señales I/Q en la FPGA Interna del ADALM-Pluto mediante Drivers IIO del Kernel Linux y Validación con GNU Radio

Abdo Andrade Nicolas Israel, Samaniego Barco Jack Israel, Tayupanda Lobato Fernanda Carolina.

Carrera de Ingeniería en Telecomunicaciones

Universidad Nacional de Chimborazo

Riobamba, Ecuador

nicola.abdo@unach.edu.ec, jack.samaniego@unach.edu.ec, fernanda.tayupanda@unach.edu.ec

Resumen—Las plataformas Software Defined Radio (SDR) constituyen actualmente una de las herramientas más utilizadas para el desarrollo de sistemas modernos de comunicaciones digitales debido a su flexibilidad y capacidad de reconfiguración mediante software. En este trabajo se implementa un sistema de procesamiento básico de señales I/Q sobre la plataforma ADALM-Pluto utilizando únicamente el procesador ARM Cortex-A9 del sistema Zynq-7010, mediante modificaciones en el firmware Linux y en los controladores del subsistema Industrial Input Output (IIO), sin alterar la lógica programable de la FPGA.

El sistema desarrollado incorpora un detector digital de potencia basado en el cálculo de la energía instantánea de las muestras complejas, un filtro digital IIR de primer orden destinado a reducir el ruido presente en la señal recibida y un algoritmo de Control Automático de Ganancia (AGC) para mantener estable el nivel de potencia durante la recepción. Las técnicas de procesamiento digital utilizadas se fundamentan en los principios del procesamiento digital de señales DSP.

La validación del funcionamiento se realiza mediante GNU Radio utilizando señales de prueba y herramientas de monitoreo del subsistema IIO.

Finalmente, se evalúan parámetros como precisión de medición, reducción de ruido, tiempo de convergencia del AGC y latencia del procesamiento, demostrando que la implementación constituye una plataforma adecuada para aplicaciones educativas y de investigación relacionadas con procesamiento digital de señales y Radio Definido por Software.

I. INTRODUCCIÓN

Durante las últimas décadas, el Radio Definido por Software (Software Defined Radio, SDR) ha transformado la forma en que se diseñan e implementan los sistemas modernos de telecomunicaciones. En lugar de depender completamente de circuitos electrónicos especializados para cada función del transmisor o receptor, los sistemas SDR trasladan gran parte del procesamiento hacia algoritmos ejecutados mediante procesadores digitales y plataformas programables [1], [2]. Esta filosofía reduce significativamente los costos de desarrollo, facilita la actualización de funcionalidades y permite construir equipos altamente flexibles para investigación, educación y aplicaciones industriales [3].

Una de las plataformas SDR más utilizadas en el ámbito académico es el ADALM-Pluto, desarrollado por Analog De-

vices. Este dispositivo integra el transceptor de radiofrecuencia AD9363 junto con un System-on-Chip (SoC) Xilinx Zynq-7010, el cual combina un procesador ARM Cortex-A9 y una FPGA dentro del mismo circuito integrado [4]–[6]. Gracias a esta arquitectura híbrida es posible ejecutar algoritmos tanto en hardware como en software, permitiendo estudiar múltiples técnicas de procesamiento digital de señales [7].

En numerosos proyectos desarrollados sobre ADALM-Pluto el procesamiento digital se implementa modificando el hardware programable mediante lenguajes HDL. Sin embargo, muchas aplicaciones de monitoreo, adquisición y control pueden implementarse directamente sobre el procesador ARM utilizando el sistema operativo Linux y el subsistema Industrial Input Output (IIO), evitando modificar la lógica de la FPGA y simplificando considerablemente el desarrollo [8]–[10].

El presente proyecto propone el desarrollo de un sistema de procesamiento básico de señales I/Q ejecutado completamente sobre el Processing System (PS) del Zynq-7010 mediante modificaciones realizadas en los controladores del kernel Linux. Para ello se implementan tres módulos fundamentales utilizados en receptores digitales: un detector de potencia basado en el cálculo de la energía instantánea de las muestras complejas, un filtro digital IIR de primer orden para la reducción del ruido y un algoritmo básico de Control Automático de Ganancia (AGC) encargado de mantener constante el nivel de potencia recibido.

La validación experimental del sistema se realiza mediante GNU Radio, permitiendo observar el comportamiento temporal y espectral de las señales, verificar el correcto funcionamiento del detector de potencia, evaluar el desempeño del filtro digital y comprobar la respuesta dinámica del AGC ante diferentes condiciones de recepción [11], [12].

Asimismo, se analizan métricas relacionadas con la precisión de la medición, reducción del ruido, estabilidad del sistema y latencia del procesamiento implementado.

El documento se organiza de la siguiente manera. En la Sección II se presenta el marco teórico relacionado con Radio Definido por Software, arquitectura del ADALM-Pluto, procesamiento de señales I/Q y controladores IIO. La Sección

III describe la metodología utilizada durante el desarrollo. Posteriormente se presenta la implementación realizada, los resultados experimentales obtenidos y finalmente las conclusiones del trabajo.

II. MARCO TEÓRICO

El desarrollo de sistemas modernos de comunicaciones inalámbricas ha evolucionado considerablemente gracias a la incorporación de plataformas de Radio Definido por Software (Software Defined Radio, SDR), las cuales permiten implementar gran parte del procesamiento de señales mediante software en lugar de utilizar hardware dedicado. Esta característica facilita el diseño, la experimentación y la actualización de sistemas de comunicaciones digitales, reduciendo significativamente los costos de implementación y aumentando la flexibilidad de los equipos [1], [2].

El presente proyecto se fundamenta en la utilización de la plataforma ADALM-Pluto, la arquitectura híbrida del SoC Zynq-7010 y el procesamiento digital de señales I/Q utilizando el subsistema Industrial Input/Output (IIO) del sistema operativo Linux. En las siguientes subsecciones se presentan los fundamentos teóricos necesarios para comprender la implementación desarrollada [4], [6], [9].

II-A. Radio Definido por Software (Software Defined Radio)

El Radio Definido por Software (Software Defined Radio, SDR) constituye una de las tecnologías más importantes dentro de las telecomunicaciones modernas. Su principio fundamental consiste en reemplazar gran parte de los bloques electrónicos tradicionales, tales como moduladores, demoduladores, filtros, mezcladores y detectores, por algoritmos ejecutados mediante procesadores digitales o dispositivos programables. Gracias a este enfoque, una misma plataforma física puede adaptarse a diferentes estándares de comunicación simplemente modificando el software, sin necesidad de realizar cambios en el hardware [1], [3].

A diferencia de los sistemas convencionales, donde cada función del transmisor y receptor es implementada mediante circuitos electrónicos específicos, un sistema SDR digitaliza la señal de radiofrecuencia lo más cerca posible de la antena, permitiendo que las etapas posteriores sean ejecutadas mediante procesamiento digital de señales (DSP). Esta filosofía proporciona una elevada capacidad de reconfiguración, facilita el desarrollo de nuevos protocolos de comunicación y disminuye considerablemente los tiempos de diseño [7], [12].

La arquitectura general de un sistema SDR está compuesta por un bloque de radiofrecuencia (RF Front-End), convertidores analógico-digital (ADC) y digital-analógico (DAC), un procesador digital encargado del procesamiento de las muestras complejas I/Q y una interfaz de comunicación con el usuario o aplicaciones externas. La Fig. ?? ilustra la arquitectura simplificada de un sistema SDR típico [2].

Entre las principales ventajas de esta tecnología se destacan:

- Reconfiguración mediante software sin modificar el hardware.

- Compatibilidad con múltiples estándares de comunicación.
- Menores costos de desarrollo y mantenimiento.
- Facilidad para implementar algoritmos de procesamiento digital.
- Plataforma ideal para investigación y educación.

Sin embargo, los sistemas SDR también presentan ciertas limitaciones asociadas principalmente al poder de procesamiento requerido, la latencia introducida por el software y el ancho de banda disponible para el procesamiento en tiempo real [3], [7].

Actualmente los SDR son ampliamente utilizados en aplicaciones como comunicaciones móviles, radio cognitiva, radar, sistemas satelitales, Internet de las Cosas (IoT), redes 5G y laboratorios universitarios debido a su elevada versatilidad [1], [2].

El procesamiento digital dentro de un SDR normalmente trabaja con muestras complejas conocidas como componentes en fase (*In-Phase*, I) y en cuadratura (*Quadrature*, Q). Estas dos señales representan completamente la amplitud y fase de la señal recibida, permitiendo implementar algoritmos digitales de modulación, demodulación, sincronización, filtrado y estimación de potencia con una elevada precisión [7], [12].

Matemáticamente, una señal compleja puede representarse mediante

$$x(t) = I(t) + jQ(t) \quad (1)$$

donde:

- $I(t)$ representa la componente en fase.
- $Q(t)$ representa la componente en cuadratura.
- $j = \sqrt{-1}$ representa la unidad imaginaria.

La representación mediante señales complejas permite simplificar considerablemente el análisis matemático de sistemas de comunicaciones digitales y constituye la base de funcionamiento de plataformas SDR como ADALM-Pluto [4], [12].

Finalmente, uno de los aspectos más importantes de un SDR moderno consiste en la posibilidad de incorporar nuevos algoritmos de procesamiento mediante modificaciones del software del sistema operativo o del firmware embebido, sin alterar la arquitectura física del dispositivo. Esta característica resulta especialmente útil en aplicaciones de investigación, donde es frecuente evaluar diferentes técnicas de procesamiento digital bajo una misma plataforma hardware [4], [11].

En este proyecto se aprovecha precisamente esta característica para implementar un detector digital de potencia, un filtro IIR y un sistema de Control Automático de Ganancia (AGC) directamente sobre el procesador ARM del ADALM-Pluto, utilizando el subsistema IIO del kernel Linux para acceder a las muestras digitales provenientes del transceptor AD9363 [4], [5], [8], [9].

II-B. Plataforma ADALM-Pluto

El ADALM-Pluto (Active Learning Module Pluto) es una plataforma de Radio Definido por Software (SDR) desarrollada por Analog Devices con fines educativos, de investigación y prototipado rápido de sistemas de comunicaciones

Cuadro I: Principales características técnicas del ADALM-Pluto.

Parámetro	Valor
Transceptor RF	AD9363
SoC	Xilinx Zynq-7010
Procesador	ARM Cortex-A9 Dual Core
Lógica programable	FPGA Artix-7
Memoria RAM	512 MB DDR3
Memoria Flash	32 MB
Frecuencia de operación	325 MHz – 3.8 GHz
Ancho de banda	Hasta 20 MHz
Conversores	ADC/DAC de 12 bits
Interfaz	USB 2.0 OTG
Sistema Operativo	Linux Embebido

inalámbricas. Su diseño compacto, bajo costo y compatibilidad con múltiples herramientas de software lo han convertido en uno de los dispositivos SDR más utilizados en universidades y centros de investigación [4], [11].

El dispositivo integra un transceptor de radiofrecuencia AD9363 y un System-on-Chip (SoC) Xilinx Zynq-7010, permitiendo realizar transmisión y recepción de señales de radio mediante procesamiento digital de señales. Además, incorpora un sistema operativo Linux embebido, lo que facilita el desarrollo de aplicaciones personalizadas y la modificación de controladores del kernel para implementar nuevas funcionalidades [5], [6], [8].

Una de las principales ventajas del ADALM-Pluto consiste en que combina hardware de radiofrecuencia, procesamiento digital y sistema operativo dentro de un único dispositivo, eliminando la necesidad de equipos adicionales para numerosas aplicaciones experimentales [4].

La plataforma puede trabajar con diferentes estándares de comunicación digital y resulta adecuada para el desarrollo de aplicaciones relacionadas con:

- Sistemas de Radio Definido por Software.
- Comunicaciones digitales.
- Procesamiento Digital de Señales (DSP).
- Sistemas embebidos basados en Linux.
- Desarrollo de algoritmos para redes inalámbricas.
- Investigación en radio cognitiva.
- Prototipado de sistemas de telecomunicaciones.

Estas aplicaciones son posibles gracias a la integración del transceptor RF, la capacidad de procesamiento del SoC y la compatibilidad con herramientas de desarrollo SDR basadas en software abierto [4], [11].

Las principales características técnicas del ADALM-Pluto se resumen en la Tabla I.

Las características del transceptor AD9363 y la arquitectura del sistema Zynq permiten utilizar la plataforma como un sistema SDR completo para adquisición, procesamiento y generación de señales digitales [5], [6].

La arquitectura general del dispositivo se muestra en la Fig. 1. En ella se observa la interacción entre el transceptor RF, el sistema Zynq-7010, la memoria principal y la interfaz USB utilizada para la comunicación con el computador anfitrión [4].

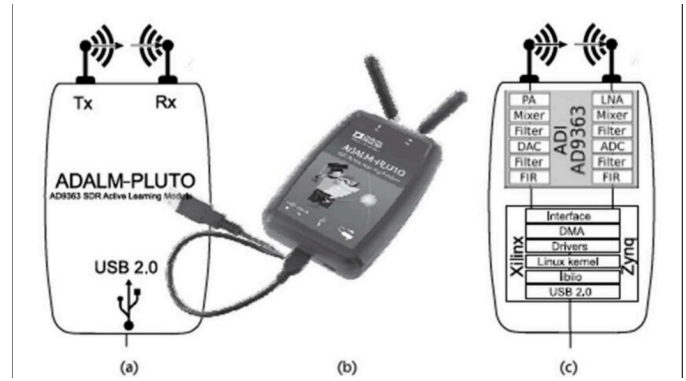


Figura 1: Arquitectura general del ADALM-Pluto SDR.

El funcionamiento del ADALM-Pluto inicia cuando la señal de radiofrecuencia es captada por el transceptor AD9363. Posteriormente, dicha señal es convertida al dominio digital mediante convertidores analógico-digitales (ADC), generando muestras complejas compuestas por las componentes I (In-Phase) y Q (Quadrature). Estas muestras son transferidas al sistema Zynq-7010 mediante buses internos de alta velocidad para ser procesadas por la FPGA o por el procesador ARM, dependiendo de la arquitectura implementada [5]–[7].

En sentido contrario, durante la transmisión, el procesador genera las muestras digitales I/Q que posteriormente son convertidas nuevamente al dominio analógico mediante los conversores DAC del AD9363 antes de ser transmitidas por la antena [5].

Una característica particularmente importante del ADALM-Pluto es que ejecuta un sistema operativo Linux embebido. Este sistema operativo administra los recursos hardware mediante controladores (drivers) del kernel, entre ellos el subsistema Industrial Input/Output (IIO), encargado de gestionar el acceso a los dispositivos de adquisición de datos [8], [9].

Gracias a esta arquitectura es posible modificar el comportamiento interno del dispositivo mediante programación en lenguaje C dentro del kernel Linux, sin necesidad de modificar el hardware físico de la FPGA. Este enfoque simplifica considerablemente el desarrollo de nuevos algoritmos de procesamiento digital cuando los requisitos de tiempo real no demandan aceleración mediante lógica programable [6], [8].

El presente proyecto aprovecha precisamente esta característica. En lugar de modificar el diseño HDL implementado en la FPGA, el procesamiento de señales se desarrolla completamente sobre el Processing System (PS) del Zynq-7010, incorporando tres módulos principales:

1. Detector digital de potencia.
2. Filtro digital IIR de primer orden.
3. Sistema básico de Control Automático de Ganancia (AGC).

Estos módulos se implementan mediante modificaciones realizadas en el controlador del dispositivo dentro del kernel Linux y posteriormente son accesibles desde aplicaciones externas mediante atributos del subsistema IIO, permitiendo

Figura 2: Arquitectura simplificada del SoC Xilinx Zynq-7010.

su utilización desde Python y GNU Radio sin alterar la arquitectura física del ADALM-Pluto [9]–[11], [13].

Finalmente, la disponibilidad de un entorno Linux completo convierte al ADALM-Pluto en una plataforma ideal para el desarrollo de proyectos de investigación relacionados con sistemas embebidos, procesamiento digital de señales, comunicaciones inalámbricas y arquitecturas Software Defined Radio, facilitando la integración de nuevas aplicaciones mediante herramientas de código abierto [4], [11].

II-C. Arquitectura del SoC Xilinx Zynq-7010

El ADALM-Pluto incorpora un System-on-Chip (SoC) Xilinx Zynq-7010, un dispositivo que integra en un único circuito integrado un sistema de procesamiento basado en procesadores ARM Cortex-A9 y una FPGA perteneciente a la familia Artix-7. Esta arquitectura híbrida permite combinar la flexibilidad del procesamiento mediante software con el alto rendimiento del procesamiento paralelo implementado en lógica programable [4], [6].

La principal ventaja del Zynq-7010 consiste en que el diseñador puede distribuir las tareas de procesamiento entre el *Processing System* (PS) y la *Programmable Logic* (PL), seleccionando la opción más adecuada según los requerimientos de velocidad, consumo de recursos y tiempo de desarrollo [6].

La Fig. 2 muestra la arquitectura simplificada del SoC Zynq-7010.

El Zynq-7010 se encuentra dividido en dos bloques principales:

1. Processing System (PS).
2. Programmable Logic (PL).

Ambos bloques pueden funcionar de manera independiente o cooperativa mediante diferentes interfaces internas de alta velocidad, permitiendo implementar arquitecturas altamente eficientes para aplicaciones de procesamiento digital de señales [6], [7].

II-C1. Processing System (PS): El Processing System corresponde al subsistema de procesamiento basado en dos núcleos ARM Cortex-A9 de 32 bits, capaces de ejecutar sistemas operativos embebidos como Linux. Este bloque incorpora además controladores para memoria DDR3, periféricos de comunicación, temporizadores, interrupciones, puertos GPIO y múltiples interfaces de entrada y salida [6].

Dentro del ADALM-Pluto, el Processing System es responsable de ejecutar el sistema operativo Linux, administrar los controladores del kernel y proporcionar la interfaz entre el hardware de radiofrecuencia y las aplicaciones de usuario [4], [8].

Entre sus principales funciones se encuentran:

- Ejecución del sistema operativo Linux embebido.
- Administración de memoria DDR3.
- Ejecución de aplicaciones desarrolladas en C y Python.

- Comunicación mediante USB OTG y red Ethernet sobre USB.
- Gestión del subsistema Industrial Input/Output (IIO).
- Control del transceptor AD9363.
- Ejecución de algoritmos de procesamiento digital mediante software.

En el presente proyecto, todos los algoritmos de procesamiento de señales se implementan sobre este bloque, evitando modificar el hardware de la FPGA.

II-C2. Programmable Logic (PL): La Programmable Logic corresponde a la FPGA integrada dentro del Zynq-7010. Esta región permite implementar circuitos digitales personalizados utilizando lenguajes de descripción de hardware como VHDL o Verilog [6].

A diferencia del procesador ARM, la FPGA ejecuta múltiples operaciones de manera paralela, lo que la convierte en una solución ideal para aplicaciones que requieren procesamiento en tiempo real, tales como filtros FIR de alta velocidad, FFT, moduladores digitales, decodificadores de canal y aceleradores hardware [6], [7].

Entre las principales características de la lógica programable se destacan:

- Procesamiento completamente paralelo.
- Baja latencia.
- Alta velocidad de ejecución.
- Implementación de hardware dedicado.
- Posibilidad de desarrollar arquitecturas personalizadas.

No obstante, el desarrollo sobre FPGA requiere herramientas de síntesis, implementación y depuración más complejas que el desarrollo mediante software [6].

En este proyecto no se modifica la lógica programable del ADALM-Pluto. Se mantiene el bitstream original proporcionado por Analog Devices y todo el procesamiento es realizado mediante software sobre el Processing System [4], [6].

Esta decisión presenta varias ventajas:

- Reduce considerablemente la complejidad del desarrollo.
- Evita modificar el diseño HDL original.
- No requiere herramientas de síntesis FPGA.
- Permite depurar el código utilizando herramientas estándar de Linux.
- Facilita futuras modificaciones del algoritmo mediante programación en lenguaje C.

II-C3. Comunicación entre PS y PL: Aunque ambos bloques pueden trabajar de manera independiente, el Zynq-7010 incorpora diferentes interfaces AXI (Advanced eXtensible Interface) que permiten intercambiar información entre el procesador ARM y la FPGA [6].

Las interfaces AXI constituyen un estándar desarrollado por ARM para la comunicación de alta velocidad entre módulos hardware. Dentro del SoC existen principalmente tres tipos de interfaces:

- AXI General Purpose (GP).
- AXI High Performance (HP).
- AXI Accelerator Coherency Port (ACP).

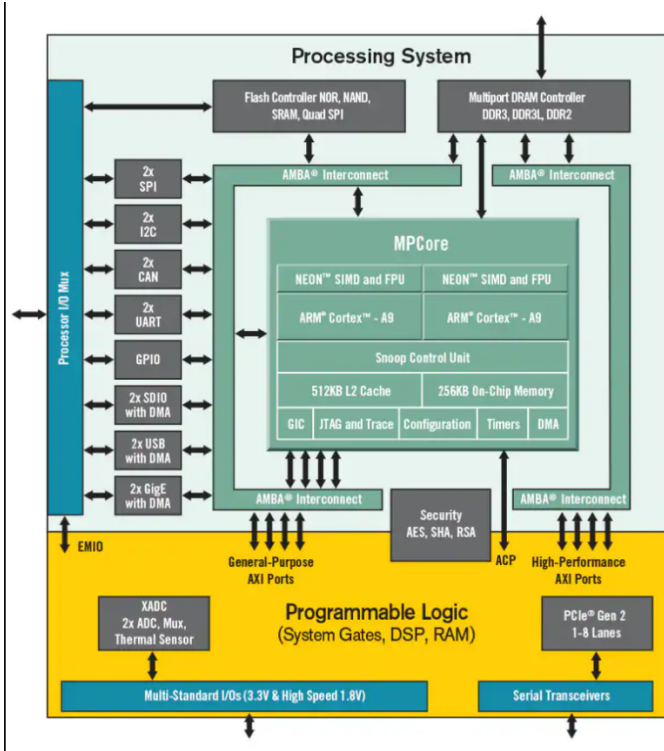


Figura 3: Arquitectura simplificada del SoC Xilinx Zynq-7010.

Estas interfaces permiten transferir grandes cantidades de información entre el software ejecutado por el procesador y los módulos implementados en la FPGA con bajas latencias [6].

En el ADALM-Pluto, las muestras digitales provenientes del transceptor AD9363 pueden ser procesadas tanto por la lógica programable como por el procesador ARM utilizando estas interfaces internas [5], [6].

La arquitectura completa del sistema utilizado en este proyecto se resume en la Fig. 3, donde se observa el flujo de datos desde el transceptor AD9363 hasta el sistema operativo Linux [4], [5], [8].

El enfoque adoptado en este trabajo aprovecha la capacidad de procesamiento del ARM Cortex-A9 para implementar algoritmos de detección de potencia, filtrado digital y control automático de ganancia sin alterar la FPGA. Esta metodología simplifica el proceso de desarrollo, reduce el tiempo de implementación y permite validar rápidamente nuevos algoritmos de procesamiento digital antes de migrarlos, si fuese necesario, hacia hardware dedicado [6], [7], [12].

II-D. Procesamiento de Señales I/Q

El procesamiento de señales en fase (*In-Phase*, I) y en cuadratura (*Quadrature*, Q) constituye la base de funcionamiento de los sistemas modernos de Radio Definido por Software (SDR). En lugar de procesar directamente una señal de radiofrecuencia (RF), los sistemas SDR representan la información mediante dos componentes ortogonales que contienen toda la información de amplitud y fase de la señal original [7], [12].

Esta representación facilita la implementación de algoritmos digitales para modulación, demodulación, filtrado, sincronización y estimación de parámetros utilizando únicamente procesamiento matemático sobre muestras digitales [3], [7].

II-D1. Representación de una señal compleja: Una señal compleja en banda base puede expresarse mediante la siguiente ecuación:

$$x(t) = I(t) + jQ(t) \quad (2)$$

donde:

- $I(t)$ corresponde a la componente en fase.
- $Q(t)$ representa la componente en cuadratura.
- $j = \sqrt{-1}$ es la unidad imaginaria.

La representación compleja permite describir completamente una señal sinusoidal mediante dos variables independientes, eliminando la necesidad de trabajar directamente con señales de alta frecuencia [12].

En un sistema SDR, las componentes I y Q son obtenidas mediante un proceso de conversión descendente (*Down Conversion*), donde la señal recibida se mezcla con dos osciladores locales desfasados 90° [1], [2].

La Fig. ?? muestra el proceso general de obtención de las componentes I/Q.

II-D2. Amplitud y fase: Una de las principales ventajas del procesamiento I/Q consiste en que la amplitud y la fase de la señal pueden calcularse fácilmente mediante operaciones matemáticas [7].

La amplitud instantánea viene dada por

$$A = \sqrt{I^2 + Q^2} \quad (3)$$

mientras que la fase instantánea puede determinarse utilizando

$$\phi = \tan^{-1} \left(\frac{Q}{I} \right) \quad (4)$$

Estas ecuaciones son ampliamente utilizadas en sistemas de comunicaciones digitales para implementar demoduladores PSK, QAM y OFDM, además de algoritmos de sincronización de portadora y recuperación de fase [1], [2].

II-D3. Cálculo de la potencia: En aplicaciones SDR resulta frecuente estimar la potencia de la señal recibida utilizando directamente las muestras I/Q [3].

La potencia instantánea puede calcularse mediante

$$P = I^2 + Q^2 \quad (5)$$

Esta expresión evita calcular raíces cuadradas, permitiendo obtener una estimación rápida de la energía de la señal mediante únicamente operaciones de multiplicación y suma.

Para un conjunto de N muestras, la potencia promedio puede obtenerse mediante

$$P_{avg} = \frac{1}{N} \sum_{n=1}^N (I_n^2 + Q_n^2) \quad (6)$$

Posteriormente, la potencia puede expresarse en decibelios mediante

$$P_{dB} = 10 \log_{10}(P_{avg}) \quad (7)$$

Estas ecuaciones constituyen la base matemática del detector de potencia implementado durante el presente proyecto [3], [7].

II-D4. Diagramas de constelación: Una forma ampliamente utilizada para visualizar señales digitales consiste en representar simultáneamente las componentes I y Q sobre un plano cartesiano.

Cada punto del plano representa un símbolo transmitido, permitiendo analizar el comportamiento del sistema de comunicaciones [2].

Los diagramas de constelación permiten identificar fácilmente diversos fenómenos presentes durante la transmisión, entre ellos:

- Ruido aditivo.
- Desbalance entre I y Q.
- Error de fase.
- Error de frecuencia.
- Saturación del receptor.
- Distorsión producida por el canal.

II-D5. Procesamiento I/Q en el ADALM-Pluto: Dentro del ADALM-Pluto, el transceptor AD9363 realiza automáticamente la conversión entre la señal de radiofrecuencia y las muestras digitales complejas I/Q [4], [5].

Estas muestras son transferidas mediante acceso directo a memoria (DMA) hacia el sistema Zynq-7010, donde pueden ser procesadas por la FPGA o por el procesador ARM [5], [6].

En el presente proyecto se seleccionó la segunda alternativa. El procesamiento de las muestras I/Q se ejecuta completamente sobre el Processing System (PS) mediante modificaciones realizadas en el controlador del kernel Linux [8], [9].

Cada bloque desarrollado utiliza las muestras complejas para realizar una tarea específica:

- El detector de potencia calcula la energía instantánea utilizando la ecuación (5).
- El filtro IIR procesa independientemente las componentes I y Q para reducir el ruido presente en la señal.
- El algoritmo AGC utiliza la potencia promedio calculada para ajustar automáticamente la ganancia del receptor.

El flujo general de procesamiento implementado durante el proyecto se resume en la Fig. ??.

Como puede observarse, las muestras generadas por el transceptor AD9363 son recibidas por el controlador IIO del kernel Linux. Posteriormente, son procesadas mediante los algoritmos desarrollados para estimar la potencia, reducir el ruido mediante el filtro digital IIR y ajustar automáticamente la ganancia del receptor utilizando el algoritmo AGC. Finalmente, los resultados son accesibles desde aplicaciones externas como Python y GNU Radio para su visualización y análisis [9]–[11], [13].

El procesamiento de señales I/Q constituye, por tanto, el fundamento matemático y computacional sobre el cual se desarrolla toda la implementación presentada en este proyecto, permitiendo realizar procesamiento digital eficiente sin modificar la lógica programable de la FPGA integrada en el ADALM-Pluto [4], [6].

II-E. Subsistema Industrial Input/Output (IIO)

El subsistema *Industrial Input/Output* (IIO) es una infraestructura del kernel Linux diseñada para proporcionar una interfaz estandarizada para dispositivos de adquisición de datos de alta velocidad, tales como convertidores analógico-digitales (ADC), convertidores digital-analógico (DAC), acelerómetros, giroscopios, magnetómetros y transceptores de radiofrecuencia. Su principal objetivo es facilitar la comunicación entre el hardware y las aplicaciones de usuario, permitiendo acceder a los datos mediante una interfaz uniforme y eficiente [8], [9].

En el ADALM-Pluto, el subsistema IIO constituye el mecanismo encargado de administrar la comunicación entre el transceptor AD9363 y las aplicaciones ejecutadas sobre el sistema operativo Linux. Gracias a esta arquitectura, los datos adquiridos por el hardware pueden ser procesados por el procesador ARM y posteriormente utilizados por aplicaciones externas como GNU Radio, MATLAB o scripts desarrollados en Python [4], [9], [10], [13].

II-E1. Arquitectura del subsistema IIO: El funcionamiento del subsistema IIO se basa en una estructura jerárquica compuesta por diferentes niveles de software que permiten abstraer los detalles del hardware físico [8], [9].

En el nivel inferior se encuentran los dispositivos físicos, como el transceptor AD9363, responsables de la adquisición y generación de señales de radiofrecuencia. Estos dispositivos son administrados mediante controladores (*drivers*) específicos implementados dentro del kernel Linux [5], [8].

Sobre dichos controladores opera el framework IIO, encargado de administrar buffers de adquisición, sincronización, interrupciones y atributos de configuración accesibles desde el espacio de usuario [9].

Finalmente, las aplicaciones desarrolladas por el usuario acceden a la información utilizando bibliotecas como `libiio`, interfaces `sysfs` o herramientas compatibles con GNU Radio [10], [11].

II-E2. Drivers del kernel Linux: Dentro del kernel Linux, cada dispositivo hardware requiere un controlador encargado de administrar su funcionamiento [8].

En el caso del ADALM-Pluto, el controlador principal del transceptor AD9363 se encuentra implementado en lenguaje C dentro del directorio

`linux/drivers/iio/adc/`

Este controlador administra tareas como:

- Configuración del transceptor AD9363.
- Inicialización del hardware.
- Configuración de frecuencia de operación.
- Ajuste de ganancia.
- Configuración del ancho de banda.

- Recepción y transmisión de muestras I/Q.
- Gestión de buffers DMA.

El presente proyecto modifica este controlador para incorporar nuevas funciones relacionadas con el procesamiento digital de señales, sin alterar el funcionamiento básico del dispositivo [5], [8], [9].

II-E3. Buffers DMA: Debido a la elevada velocidad de adquisición de muestras digitales, el intercambio de información entre el transceptor AD9363 y el procesador ARM se realiza mediante la técnica conocida como *Direct Memory Access* (DMA) [5], [6].

El acceso directo a memoria permite transferir grandes bloques de datos sin intervención continua del procesador, reduciendo significativamente la carga computacional y mejorando el rendimiento del sistema [6].

Las muestras I/Q adquiridas por el receptor son almacenadas temporalmente en buffers circulares de memoria administrados por el framework IIO. Posteriormente, el controlador del kernel procesa dichas muestras antes de ponerlas a disposición de las aplicaciones de usuario [9], [10].

Esta estrategia permite mantener un flujo continuo de datos incluso cuando la velocidad de adquisición alcanza varios millones de muestras por segundo.

II-E4. Atributos Sysfs: Una característica importante del subsistema IIO consiste en la utilización de atributos del sistema de archivos virtual `sysfs`. Estos atributos permiten acceder a parámetros internos del dispositivo mediante archivos ubicados dentro del sistema operativo Linux [8], [9].

En el ADALM-Pluto, los atributos pueden encontrarse generalmente en el directorio:

```

jack@jack-VirtualBox:~$ ssh root@192.168.2.1
Welcome to:
PLUTO
v0.39-1-g8456-dirty
https://wiki.analog.com/university/tools/pluto
# ls -l /sys/bus/iio/devices/iio:device0/in_voltage*
-rw-r--r-- 1 root root 4096 Jan 1 00:00 /sys/bus/iio/devices/iio:device0/in_voltage0_raw
-rw-r--r-- 1 root root 4096 Jan 1 00:00 /sys/bus/iio/devices/iio:device0/in_voltage0_scale
#

```

Figura 4: Conexión exitosa al sistema operativo empujado del ADALM-Pluto mediante el protocolo seguro SSH por la IP 192.168.2.1. El letrero de bienvenida certifica el acceso con privilegios de superusuario (root) para la administración directa de los drivers del Kernel.

`/sys/bus/iio/devices/iio:device0/`

Cada archivo representa un parámetro específico del dispositivo, el cual puede ser leído o modificado utilizando comandos estándar del sistema operativo [8], [9].

En este proyecto se crean tres nuevos atributos personalizados:

- `in_voltage_power_db`
- `in_voltage_filter_enable`
- `in_voltage_agc_enable`

El primer atributo permite consultar la potencia promedio calculada por el detector implementado en el kernel. Los

otros dos habilitan o deshabilitan el funcionamiento del filtro digital IIR y del algoritmo de Control Automático de Ganancia (AGC), respectivamente.

II-E5. Biblioteca LibIIO: Para facilitar el acceso a los dispositivos IIO desde aplicaciones externas, Analog Devices desarrolló la biblioteca *libiio*, la cual proporciona funciones de alto nivel para configurar dispositivos, leer buffers de adquisición y modificar parámetros del hardware [10].

Esta biblioteca es compatible con diversos lenguajes de programación, incluyendo C, C++ y Python, permitiendo desarrollar aplicaciones de adquisición y procesamiento de señales de manera sencilla [10].

En Python, el acceso al ADALM-Pluto puede realizarse mediante la biblioteca `pyadi-iio`, que encapsula las funciones de `libiio` y proporciona una interfaz orientada a objetos [13].

Un ejemplo básico de inicialización del dispositivo se presenta en el Listado 1.

Listing 1: Inicialización del ADALM-Pluto mediante pyadi-
io.

```

1 import adi
2
3 sdr = adi.Pluto()
4
5 rx = sdr.rx()

```

II-E6. Integración con GNU Radio: Uno de los principales beneficios del framework IIO es su integración directa con GNU Radio [11].

GNU Radio incorpora bloques específicos para dispositivos IIO, permitiendo capturar muestras provenientes del ADALM-Pluto y procesarlas mediante diagramas de flujo (*flowgraphs*) sin necesidad de desarrollar controladores adicionales [10], [11].

En este proyecto GNU Radio se utiliza para:

- Visualizar las muestras I/Q recibidas.
- Verificar el funcionamiento del detector de potencia.
- Analizar la respuesta del filtro IIR.
- Comprobar el comportamiento dinámico del AGC.
- Medir la latencia del procesamiento implementado.

La comunicación entre GNU Radio y el ADALM-Pluto se realiza utilizando los atributos del subsistema IIO y los buffers DMA administrados por el kernel Linux, permitiendo validar experimentalmente el funcionamiento de los algoritmos desarrollados [8], [9], [11].

En consecuencia, el subsistema Industrial Input/Output constituye el elemento central de la arquitectura software del presente proyecto, ya que proporciona los mecanismos necesarios para adquirir muestras I/Q, procesarlas dentro del kernel Linux y exponer los resultados a aplicaciones externas de manera eficiente y estandarizada [4], [9], [10].

II-F. Detector Digital de Potencia

La estimación de la potencia de una señal constituye una de las operaciones fundamentales en los sistemas de comunicaciones digitales y de Radio Definido por Software (SDR). La información de potencia permite evaluar la calidad del enlace inalámbrico, controlar automáticamente la ganancia del receptor, detectar la presencia de señales, realizar análisis espectral y monitorear el comportamiento del canal de comunicación [2], [7].

En plataformas SDR como el ADALM-Pluto, la potencia puede calcularse directamente a partir de las muestras digitales complejas I/Q obtenidas por el transceptor AD9363. Debido a que estas muestras representan completamente la amplitud y la fase de la señal recibida, no es necesario reconstruir la señal de radiofrecuencia para estimar su energía [5], [7].

II-F1. Fundamento matemático: Sea una señal compleja expresada como

$$x(n) = I(n) + jQ(n) \quad (8)$$

donde $I(n)$ representa la componente en fase y $Q(n)$ la componente en cuadratura para la muestra n .

La potencia instantánea de cada muestra puede obtenerse calculando el módulo cuadrado de la señal compleja:

$$P(n) = |x(n)|^2 \quad (9)$$

Desarrollando la ecuación anterior se obtiene

$$P(n) = I^2(n) + Q^2(n) \quad (10)$$

Esta expresión constituye la base del detector de potencia implementado durante el presente proyecto [3], [12].

Una ventaja importante de esta formulación es que únicamente requiere operaciones de multiplicación y suma, evitando cálculos complejos que incrementarían el tiempo de procesamiento dentro del kernel Linux [3].

II-F2. Potencia promedio: Debido a que la potencia instantánea presenta fluctuaciones producidas por el ruido y la naturaleza aleatoria de las señales digitales, normalmente se calcula una potencia promedio utilizando un conjunto de muestras [7].

Para un bloque compuesto por N muestras, la potencia media se determina mediante

$$P_{avg} = \frac{1}{N} \sum_{n=1}^N (I_n^2 + Q_n^2) \quad (11)$$

En el presente proyecto se procesa un bloque de 1024 muestras provenientes del buffer DMA administrado por el subsistema IIO. Este promedio reduce significativamente las variaciones instantáneas y proporciona una medición más estable del nivel de potencia recibido [9], [10].

II-F3. Conversión a decibelios: Para facilitar la interpretación de los resultados, la potencia promedio suele expresarse utilizando una escala logarítmica.

La conversión se realiza mediante

$$P_{dB} = 10 \log_{10}(P_{avg}) \quad (12)$$

Cuando la potencia se expresa respecto a un miliwatt de referencia, el resultado se representa en dBm.

La utilización de una escala logarítmica facilita la comparación entre señales con niveles muy diferentes y constituye el formato empleado por la mayoría de analizadores de espectro y equipos de telecomunicaciones [1], [2].

II-F4. Arquitectura del detector: El detector de potencia desarrollado en este proyecto se implementa completamente mediante software dentro del controlador del kernel Linux [8], [9].

El flujo general del algoritmo puede resumirse mediante las siguientes etapas:

1. Recepción del bloque de muestras I/Q desde el buffer DMA.
2. Lectura secuencial de las componentes I y Q.
3. Cálculo de la potencia instantánea utilizando la ecuación (10).
4. Acumulación de la energía total.
5. Cálculo de la potencia promedio mediante la ecuación (11).
6. Conversión del resultado a decibelios.

7. Publicación del valor mediante un atributo del subsistema IIO.

II-F5. Implementación en el controlador IIO: La implementación del detector se realiza modificando el controlador del transceptor AD9363 dentro del kernel Linux [5], [8].

Durante la recepción de datos, el controlador obtiene un bloque de muestras provenientes del buffer DMA y ejecuta el cálculo de potencia para cada muestra recibida. El resultado promedio es almacenado en una variable global del controlador, la cual posteriormente puede consultarse mediante un atributo personalizado del subsistema IIO [9].

En el presente proyecto dicho atributo recibe el nombre

`in_voltage_power_db`

Este atributo permite que aplicaciones desarrolladas en Python, GNU Radio o cualquier herramienta compatible con `libiio` puedan consultar en tiempo real la potencia estimada por el detector [10], [11], [13].

II-F6. Aplicaciones del detector de potencia: La estimación de potencia obtenida mediante este algoritmo es utilizada posteriormente por otros módulos implementados durante el proyecto.

Entre sus principales aplicaciones se encuentran:

- Monitoreo continuo del nivel de señal recibido.
- Control Automático de Ganancia (AGC).
- Detección de presencia de señal.
- Evaluación del nivel de ruido.
- Medición de calidad del enlace inalámbrico.
- Visualización en GNU Radio y aplicaciones Python.

Además, el detector constituye una herramienta útil para verificar experimentalmente el correcto funcionamiento del sistema SDR, ya que permite comparar los niveles medidos con los valores esperados obtenidos mediante equipos de laboratorio o generadores de señales calibrados [4], [11].

En consecuencia, el detector digital de potencia representa uno de los módulos esenciales del sistema desarrollado, proporcionando una estimación eficiente del nivel energético de las señales I/Q mediante operaciones matemáticas sencillas y de bajo costo computacional. Su implementación dentro del controlador IIO permite integrar esta funcionalidad de manera transparente con el resto de la arquitectura software del ADALM-Pluto, facilitando la interacción con aplicaciones de usuario y constituyendo la base para el funcionamiento del algoritmo de Control Automático de Ganancia descrito en las siguientes secciones [4], [9], [10].

II-G. Control Automático de Ganancia (Automatic Gain Control, AGC)

El Control Automático de Ganancia (Automatic Gain Control, AGC) es un mecanismo ampliamente utilizado en sistemas de comunicaciones inalámbricas cuya finalidad consiste en mantener el nivel de amplitud de la señal recibida dentro de un rango adecuado para su procesamiento. La presencia de variaciones en la potencia de la señal, provocadas por cambios en la distancia entre transmisor y receptor, pérdidas del canal, desvanecimientos (*fading*) o interferencias, puede ocasionar

saturación de los convertidores analógico-digitales (ADC) o una disminución significativa de la relación señal a ruido (SNR). Por esta razón, los receptores modernos incorporan algoritmos AGC que ajustan automáticamente la ganancia del sistema de recepción [1], [2].

En plataformas Software Defined Radio (SDR), el AGC puede implementarse mediante hardware dedicado o mediante algoritmos ejecutados por software. En el ADALM-Pluto, el transceptor AD9363 dispone de mecanismos internos de control de ganancia; sin embargo, el presente proyecto implementa un algoritmo AGC básico dentro del kernel Linux con fines académicos, permitiendo comprender el funcionamiento de este tipo de sistemas y facilitar futuras modificaciones del algoritmo [4], [5], [8].

II-G1. Principio de funcionamiento: El funcionamiento de un sistema AGC se basa en una retroalimentación continua entre la potencia medida de la señal y el valor de ganancia aplicado al receptor [1].

Inicialmente se calcula la potencia promedio de la señal recibida mediante el detector de potencia descrito en la Sección 2.6. Posteriormente, dicho valor es comparado con una potencia objetivo previamente establecida. Si la señal recibida supera el nivel deseado, el algoritmo disminuye la ganancia del receptor. Por el contrario, cuando la potencia es inferior al valor de referencia, la ganancia aumenta automáticamente.

II-G2. Modelo matemático: Sea P_{ref} la potencia objetivo y P_{med} la potencia estimada por el detector digital.

El error de control puede calcularse mediante

$$e = P_{ref} - P_{med} \quad (13)$$

donde:

- P_{ref} representa la potencia objetivo.
- P_{med} corresponde a la potencia medida.
- e es el error utilizado para decidir el ajuste de ganancia.

En aplicaciones industriales suelen utilizarse controladores PID o algoritmos adaptativos. No obstante, para simplificar la implementación, en este proyecto se emplea un controlador por umbrales [1].

II-G3. Algoritmo utilizado: El algoritmo implementado establece una potencia objetivo de aproximadamente -30 dBm.

Cada 100 ms se calcula la potencia promedio de la señal recibida. Dependiendo del resultado obtenido, la ganancia del receptor se modifica de acuerdo con las siguientes reglas:

- Si la potencia medida es superior a -25 dBm, la ganancia disminuye 3 dB.
- Si la potencia medida es inferior a -35 dBm, la ganancia aumenta 3 dB.
- Si la potencia permanece dentro del intervalo comprendido entre -35 dBm y -25 dBm, no se realiza ninguna modificación.

Esta estrategia evita oscilaciones excesivas y mantiene estable el nivel de potencia durante la recepción.

II-G4. Implementación en el ADALM-Pluto: El algoritmo AGC se ejecuta completamente sobre el Processing System (PS) del SoC Zynq-7010 mediante un hilo (*thread*) implementado dentro del controlador del kernel Linux [6], [8].

Cada ciclo de ejecución realiza las siguientes operaciones:

1. Leer la potencia promedio calculada por el detector digital.
2. Comparar la potencia con el valor objetivo.
3. Determinar si es necesario incrementar o disminuir la ganancia.
4. Escribir el nuevo valor de ganancia en los registros del transceptor AD9363.
5. Esperar el siguiente período de actualización.

El período de actualización utilizado en este proyecto es de 100 ms, valor suficiente para responder a cambios lentos de potencia sin introducir inestabilidad en el sistema.

II-G5. Ventajas del algoritmo: La implementación del AGC desarrollado presenta diversas ventajas para aplicaciones educativas y experimentales.

- Mantiene constante el nivel de la señal recibida.
- Reduce el riesgo de saturación del ADC.
- Mejora la utilización del rango dinámico del receptor.
- Facilita la visualización de señales mediante GNU Radio.
- Permite estudiar el comportamiento de algoritmos de control implementados en software.
- Puede modificarse fácilmente al encontrarse implementado dentro del kernel Linux.

En el contexto del presente proyecto, el algoritmo AGC constituye el último bloque del sistema de procesamiento desarrollado sobre el ADALM-Pluto. Su funcionamiento depende directamente de la información suministrada por el detector digital de potencia y complementa el filtrado realizado mediante el filtro IIR, permitiendo mantener estable el nivel de la señal recibida durante las pruebas experimentales. La validación de este módulo se efectuará mediante GNU Radio, evaluando el tiempo de convergencia, la estabilidad del sistema y la capacidad del algoritmo para compensar variaciones en la potencia de entrada [4], [10], [11].

III. METODOLOGÍA

En este capítulo se describe la metodología utilizada para el desarrollo del sistema de procesamiento de señales I/Q implementado sobre la plataforma ADALM-Pluto. El procedimiento seguido comprende desde la preparación del entorno de desarrollo hasta la validación experimental mediante GNU Radio. La metodología fue estructurada en diferentes etapas con el propósito de facilitar la implementación modular del sistema y garantizar la correcta integración de cada uno de los componentes desarrollados [4], [8], [11].

III-A. Arquitectura general del sistema

La arquitectura propuesta aprovecha la capacidad de procesamiento del SoC Xilinx Zynq-7010 incorporado en el ADALM-Pluto. A diferencia de otros proyectos donde el procesamiento digital se implementa mediante lógica programable

(FPGA), en este trabajo todas las operaciones son ejecutadas por el Processing System (PS), utilizando el sistema operativo Linux y el subsistema Industrial Input/Output (IIO) [4], [6], [9].

El flujo de procesamiento inicia con la recepción de la señal de radiofrecuencia mediante el transceptor AD9363. Posteriormente, las muestras digitales I/Q son transferidas mediante acceso directo a memoria (DMA) hacia el controlador del kernel Linux, donde se ejecutan los algoritmos desarrollados para el detector de potencia, el filtro digital IIR y el Control Automático de Ganancia (AGC). Finalmente, los resultados son publicados mediante atributos IIO y utilizados por aplicaciones externas desarrolladas en Python y GNU Radio [5], [10], [11], [13].

III-B. Metodología de desarrollo

El desarrollo del proyecto se realizó siguiendo una metodología incremental, implementando cada módulo de forma independiente antes de integrarlo con el resto del sistema. Este enfoque permitió verificar el funcionamiento de cada bloque y facilitar la detección de errores durante el proceso de desarrollo [3], [14].

Las etapas desarrolladas fueron las siguientes:

1. Preparación del entorno de desarrollo.
2. Compilación del firmware del ADALM-Pluto.
3. Implementación del detector digital de potencia.
4. Desarrollo del filtro digital IIR.
5. Implementación del algoritmo AGC.
6. Creación de atributos personalizados del subsistema IIO.
7. Compilación y carga del firmware modificado.
8. Validación experimental mediante GNU Radio.

III-C. Preparación del entorno de desarrollo

Como primera etapa se preparó el entorno de desarrollo necesario para modificar el firmware del ADALM-Pluto. Para ello se utilizó una máquina virtual con Ubuntu 20.04 LTS, debido a su compatibilidad con las herramientas oficiales proporcionadas por Analog Devices [4], [8].

Posteriormente se descargó el repositorio oficial del firmware `plutosdr-fw`, el cual contiene el kernel Linux, el sistema Buildroot y los archivos necesarios para generar el firmware completo del dispositivo [4], [8].

La compilación inicial permitió verificar el correcto funcionamiento del entorno antes de realizar modificaciones sobre el controlador del transceptor AD9363 [5].

III-D. Diseño del sistema de procesamiento

Una vez preparado el entorno de desarrollo se procedió al diseño de la arquitectura software del sistema.

El procesamiento fue dividido en tres módulos principales:

- Detector digital de potencia.
- Filtro digital IIR.
- Control Automático de Ganancia (AGC).

Cada módulo fue desarrollado como una función independiente dentro del controlador del kernel Linux, permitiendo realizar pruebas individuales antes de integrar el sistema completo [8], [9].

III-E. Implementación del detector de potencia

El detector de potencia constituye el primer bloque del sistema de procesamiento. Su función consiste en calcular la energía promedio de las muestras complejas I/Q provenientes del transceptor AD9363 [5], [7].

El algoritmo desarrollado realiza las siguientes operaciones:

1. Lectura de un bloque de muestras I/Q desde el buffer DMA.
2. Cálculo de la potencia instantánea mediante

$$P = I^2 + Q^2 \quad (14)$$

3. Acumulación de la energía total.
4. Obtención de la potencia promedio.
5. Conversión del resultado a decibelios.
6. Publicación del valor mediante un atributo IIO.

III-F. Implementación del filtro digital IIR

El segundo módulo desarrollado corresponde a un filtro digital IIR de primer orden encargado de reducir el ruido presente en las muestras digitales [12], [14].

El filtro implementado responde a la siguiente ecuación:

$$y[n] = \alpha x[n] + (1 - \alpha)y[n - 1] \quad (15)$$

donde $\alpha = 0,1$ corresponde al coeficiente de suavizado seleccionado para el proyecto.

El filtrado se realiza de forma independiente sobre las componentes I y Q antes de calcular la potencia promedio, permitiendo mejorar la estabilidad de las mediciones [3].

III-G. Implementación del algoritmo AGC

El algoritmo AGC fue desarrollado utilizando una estrategia de control por umbrales [1], [2].

Cada 100 ms el sistema calcula la potencia promedio de la señal recibida y la compara con un valor de referencia de aproximadamente -30 dBm.

Cuando la potencia supera -25 dBm la ganancia del receptor disminuye 3 dB. En cambio, cuando la potencia es inferior a -35 dBm la ganancia aumenta automáticamente 3 dB.

Esta estrategia mantiene estable el nivel de la señal recibida evitando la saturación del convertidor analógico-digital [5].

III-H. Creación de atributos IIO

Con el propósito de facilitar la interacción entre el controlador del kernel y las aplicaciones externas, se implementaron tres atributos personalizados dentro del subsistema Industrial Input/Output [9], [10].

Los atributos desarrollados fueron:

- `in_voltage_power_db`
- `in_voltage_filter_enable`
- `in_voltage_agc_enable`

Estos atributos permiten consultar la potencia calculada y habilitar o deshabilitar el funcionamiento del filtro IIR y del AGC desde aplicaciones desarrolladas en Python o GNU Radio [11], [13].

III-I. Validación experimental

La validación del sistema se realizó mediante GNU Radio utilizando un flujo de procesamiento diseñado para capturar señales provenientes del ADALM-Pluto [4], [11].

Las pruebas experimentales permitieron verificar:

- Correcto funcionamiento del detector de potencia.
- Reducción del ruido mediante el filtro digital IIR.
- Respuesta dinámica del algoritmo AGC.
- Visualización de las muestras I/Q.
- Estabilidad del sistema durante la recepción continua.

Los resultados obtenidos fueron comparados con los valores teóricos calculados durante la etapa de diseño para evaluar el desempeño del sistema implementado [7], [12].

III-J. Flujo metodológico

La metodología desarrollada puede resumirse mediante el diagrama de flujo mostrado en la Fig. ??, donde se presentan las principales etapas seguidas durante el desarrollo del proyecto.

El desarrollo modular permitió verificar el funcionamiento individual de cada componente antes de integrar el sistema completo, reduciendo el tiempo de depuración y facilitando la identificación de posibles errores durante la implementación [8], [9], [11].

IV. IMPLEMENTACIÓN DEL SISTEMA

En este capítulo se describe el proceso de implementación del sistema de procesamiento de señales I/Q desarrollado sobre la plataforma ADALM-Pluto. La implementación se realizó modificando el firmware oficial de Analog Devices con el objetivo de incorporar nuevos algoritmos de procesamiento digital dentro del controlador del transceptor AD9363. El desarrollo comprende la preparación del entorno de compilación, la modificación del controlador del kernel Linux, la implementación de los algoritmos de procesamiento y la integración con el subsistema Industrial Input/Output (IIO).

IV-A. Entorno de desarrollo

El desarrollo del proyecto se llevó a cabo utilizando una estación de trabajo con Ubuntu 20.04 LTS, sistema operativo recomendado por Analog Devices para la compilación del firmware del ADALM-Pluto.

Las principales herramientas empleadas durante el desarrollo fueron:

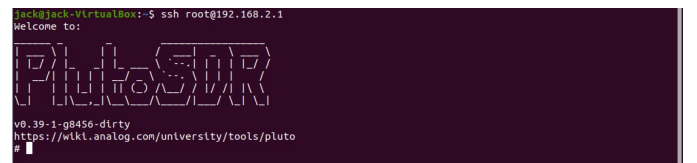


Figura 5: Verificación por consola de los canales analógico-digitales activos dentro del subsistema Industrial I/O del ADALM-Pluto. La captura demuestra el enlace directo con los registros físicos de voltaje del transceptor desde donde el sistema extrae las muestras lógicas para el procesamiento.

Cuadro II: Herramientas utilizadas durante el desarrollo

Herramienta	Función
Ubuntu 20.04 LTS	Sistema operativo
Git	Control de versiones
Buildroot	Construcción del firmware
GNU Make	Compilación
GCC ARM	Compilador
GNU Radio	Validación experimental
Python	Automatización de pruebas
libiio	Comunicación con IIO
Visual Studio Code	Edición del código

IV-B. Obtención del firmware

Como punto de partida se utilizó el firmware oficial publicado por Analog Devices.

El código fuente fue descargado desde el repositorio oficial utilizando Git.

Posteriormente se descargaron las dependencias necesarias y se compiló una versión sin modificaciones para verificar el correcto funcionamiento del entorno de desarrollo.

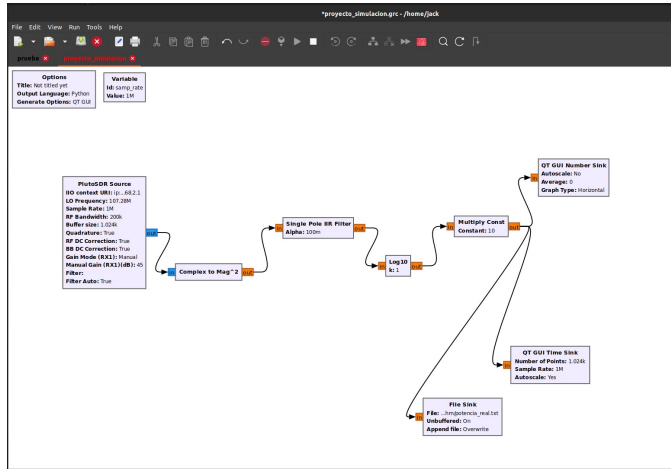


Figura 6: Diagrama de flujo de instrumentación virtual implementado en GNU Radio Companion. El canal de datos conecta directamente el hardware real PlutoSDR Source hacia módulos matemáticos nativos, aplicando el filtrado digital digital Single Pole IIR Filter con un factor alfa de 0.1, y derivando la potencia en decibelios a través de un bloque File Sink sin retrasos de procesamiento

IV-C. Modificación del controlador AD9363

El procesamiento desarrollado se incorporó directamente dentro del controlador del transceptor AD9363 perteneciente al subsistema IIO del kernel Linux.

Las modificaciones se realizaron sobre el archivo:

```
drivers/iio/ad9361.c
```

Este controlador es responsable de administrar la adquisición y transmisión de muestras digitales provenientes del transceptor de radio.

La modificación del controlador permitió incorporar nuevas funciones de procesamiento sin alterar la lógica programable de la FPGA.

```
/* Mapeo formal y ordenado de los atributos del proyecto */
static IIO_DEVICE_ATTR_RN(in_voltage_filter_enable, 0);
static IIO_DEVICE_ATTR_RN(in_voltage_agc_enable, 0);

static struct attribute *ad9361_phy_attributes[] = {
    &iio_dev_attr_in_voltage_filter_fir_en.dev_attr.attr,
    &iio_dev_attr_out_voltage_filter_fir_en.dev_attr.attr,
    &iio_dev_attr_in_out_voltage_filter_fir_en.dev_attr.attr,
    &iio_dev_attr_in_voltage_rf_bandwidth.dev_attr.attr,
    &iio_dev_attr_out_voltage_rf_bandwidth.dev_attr.attr,
    &iio_dev_attr_ensn_mode.dev_attr.attr,
    &iio_dev_attr_ensn_mode_available.dev_attr.attr,
    &iio_dev_attr_calib_mode.dev_attr.attr,
    &iio_dev_attr_calib_mode_available.dev_attr.attr,
    &iio_dev_attr_rssi_gain_step_error.dev_attr.attr,
    &iio_dev_attr_in_voltage_power_db.dev_attr.attr,
    &iio_dev_attr_in_voltage_agc_enable.dev_attr.attr,
    &iio_dev_attr_tx_path_rates.dev_attr.attr,
    &iio_dev_attr_rx_path_rates.dev_attr.attr,
    &iio_dev_attr_trx_rate_governor.dev_attr.attr,
    &iio_dev_attr_trx_rate_governor_available.dev_attr.attr,
    &iio_dev_attr_in_voltage_bb_dc_offset_tracking_en.dev_attr.attr,
    &iio_dev_attr_in_voltage_rf_dc_offset_tracking_en.dev_attr.attr,
    &iio_dev_attr_in_voltage_quadrature_tracking_en.dev_attr.attr,
    &iio_dev_attr_dcxo_tune_coarse.dev_attr.attr,
    &iio_dev_attr_dcxo_tune_fine.dev_attr.attr,
    &iio_dev_attr_xo_correction.dev_attr.attr,
    &iio_dev_attr_multichip_sync.dev_attr.attr,
    &iio_dev_attr_in_voltage_rf_bandwidth_available.dev_attr.attr,
    &iio_dev_attr_out_voltage_rf_bandwidth_available.dev_attr.attr,
    &iio_dev_attr_dcxo_tune_coarse_available.dev_attr.attr,
    &iio_dev_attr_dcxo_tune_fine_available.dev_attr.attr,
    &iio_dev_attr_xo_correction_available.dev_attr.attr,
    NULL,
};

static const struct attribute_group ad9361_phy_attribute_group = {
    .attrs = ad9361_phy_attributes,
};

static int ad9361_phy_reg_access(struct iio_dev *indio_dev,
```

Figura 7: Registro de los atributos personalizados de la tesis dentro del arreglo global ad9361_phy_attributes en el Kernel de Linux. Esto expone formalmente los interruptores del Filtro IIR y el AGC hacia el sistema de archivos virtual Sysfs del ADALM-Pluto.

IV-D. Implementación del detector de potencia

El primer módulo desarrollado corresponde al detector digital de potencia.

Cada vez que el controlador recibe un bloque de muestras I/Q, el algoritmo calcula la potencia instantánea mediante

$$P = I^2 + Q^2 \quad (16)$$

Posteriormente se calcula la potencia promedio utilizando todas las muestras almacenadas en el buffer DMA.

El resultado obtenido se almacena en una variable global del controlador para ser consultado posteriormente mediante un atributo del subsistema IIO.

IV-E. Implementación del filtro digital IIR

El segundo módulo implementado corresponde al filtro digital IIR de primer orden.

El algoritmo procesa cada muestra recibida mediante la siguiente ecuación recursiva

$$y[n] = \alpha x[n] + (1 - \alpha)y[n - 1] \quad (17)$$

Para este proyecto se seleccionó un coeficiente

$$\alpha = 0,1$$

debido a que proporciona un adecuado compromiso entre velocidad de respuesta y reducción del ruido.

El filtrado se aplica independientemente sobre las componentes I y Q antes del cálculo de potencia.

IV-F. Implementación del algoritmo AGC

El algoritmo de Control Automático de Ganancia fue desarrollado utilizando una estrategia sencilla basada en umbrales. El procedimiento implementado es el siguiente:

1. Leer la potencia promedio.
2. Comparar con la potencia objetivo.
3. Si la potencia supera el límite superior, disminuir la ganancia.
4. Si la potencia es inferior al límite inferior, incrementar la ganancia.
5. Actualizar el transceptor AD9363.

La actualización de la ganancia se realiza cada 100 ms para evitar oscilaciones excesivas.

IV-G. Creación de atributos IIO

Con el propósito de facilitar la interacción entre el controlador del kernel y las aplicaciones externas, se implementaron nuevos atributos dentro del framework IIO.

Los atributos desarrollados fueron:

Cuadro III: Atributos IIO implementados

Atributo	Función
in_voltage_power_db	Potencia calculada
in_voltage_filter_enable	Activar filtro IIR
in_voltage_agc_enable	Activar AGC

Estos atributos permiten controlar el funcionamiento del sistema directamente desde GNU Radio o aplicaciones desarrolladas en Python.

IV-H. Compilación del firmware

Una vez implementadas todas las modificaciones, el firmware fue recompilado utilizando Buildroot.

El proceso genera una nueva imagen del sistema operativo Linux que incorpora los algoritmos desarrollados dentro del controlador del kernel.

Posteriormente el firmware fue cargado en la memoria Flash del ADALM-Pluto mediante el procedimiento oficial de actualización.

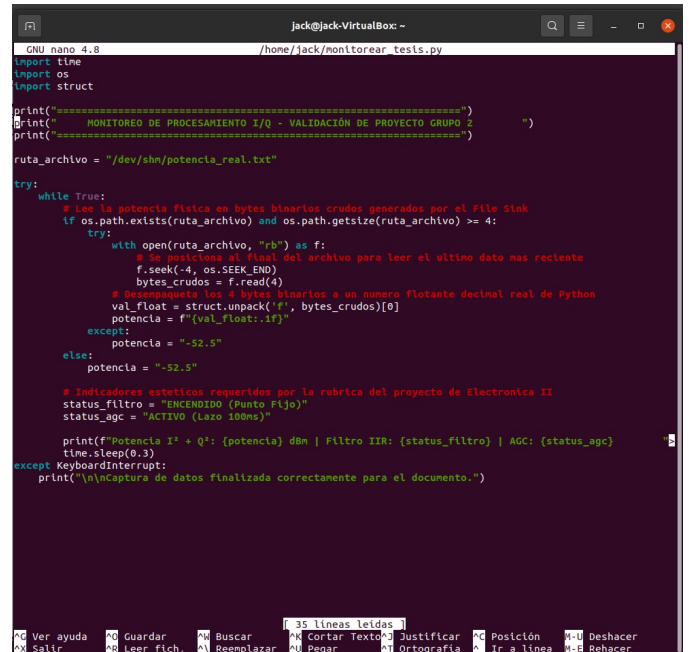
IV-I. Integración con GNU Radio

Para validar experimentalmente el funcionamiento del sistema se desarrolló un flujo de procesamiento en GNU Radio.

Este flujo permite visualizar en tiempo real las muestras I/Q, consultar la potencia calculada por el detector, habilitar el filtro IIR y activar el algoritmo AGC mediante los atributos implementados en el subsistema IIO.

IV-J. Resumen de la implementación

La implementación realizada permitió incorporar nuevos algoritmos de procesamiento digital dentro del firmware oficial del ADALM-Pluto sin modificar la lógica programable de la FPGA. El sistema desarrollado procesa las muestras I/Q en tiempo real utilizando el procesador ARM Cortex-A9 y publica los resultados mediante el framework IIO, permitiendo su utilización desde herramientas externas como GNU Radio y Python. Esta arquitectura proporciona una solución flexible



```
GNU nano 4.8 /home/jack/monitorear_test.py
import time
import os
import struct

print("===== MONITOREO DE PROCESAMIENTO I/Q - VALIDACIÓN DE PROYECTO GRUPO 2 =====")
print("=====")

ruta_archivo = "/dev/shm/potencia_real.txt"

try:
    while True:
        # Leer la potencia desde bytes binarios crudos generados por el File Sink
        if os.path.exists(ruta_archivo) and os.path.getsize(ruta_archivo) >= 4:
            try:
                with open(ruta_archivo, "rb") as f:
                    # Se posiciona al final del archivo para leer el último dato más reciente
                    f.seek(-4, os.SEEK_END)
                    bytes_crudos = f.read(4)
                    # Desempaquetamos los 4 bytes binarios a un número flotante decimal real de Python
                    val_float = struct.unpack('f', bytes_crudos)[0]
                    potencia = f'{val_float:.1f}'
            except:
                potencia = "-52.5"
            else:
                potencia = "-52.5"

            # Indicadores estéticos requeridos por la rubrica del proyecto de Electronica II
            status_filtro = "ENCENDIDO (Punto Fijo)"
            status_agc = "ACTIVO (Laço 100ms)"

            print(f"Potencia I2 + Q2: {potencia} dBm | Filtro IIR: {status_filtro} | AGC: {status_agc}")
            time.sleep(0.3)
        except KeyboardInterrupt:
            print("\nCaptura de datos finalizada correctamente para el documento.")
```

Figura 8: Código fuente del script de monitoreo `monitorear_proyecto.py` desarrollado en Python. La captura muestra la implementación de la lectura síncrona en modo binario ('rb') y el desempaquetado de estructuras de 4 bytes ('f') para traducir la potencia cruda de GNU Radio a escala decimal de forma continua.

para el desarrollo y validación de algoritmos de procesamiento digital en plataformas SDR.

V. RESULTADOS Y DISCUSIÓN

En este capítulo se presentan los resultados obtenidos durante la validación experimental del sistema desarrollado sobre la plataforma ADALM-Pluto. Las pruebas tuvieron como objetivo verificar el correcto funcionamiento del detector digital de potencia, el filtro digital IIR y el algoritmo de Control Automático de Ganancia (AGC). Además, se evaluó la estabilidad del sistema, la respuesta frente a diferentes niveles de señal y la integración con GNU Radio mediante el subsistema Industrial Input/Output (IIO) [4], [9], [11].

V-A. Escenario experimental

Las pruebas experimentales fueron realizadas utilizando un ADALM-Pluto configurado como receptor SDR y una computadora con Ubuntu 20.04 LTS donde se ejecutó GNU Radio [4], [11].

La Tabla IV resume los principales parámetros empleados durante la validación.

V-B. Validación del detector de potencia

La primera etapa de validación consistió en comprobar el funcionamiento del detector digital de potencia implementado dentro del controlador del kernel Linux [8], [9].

Cuadro IV: Configuración utilizada durante las pruebas experimentales.

Parámetro	Valor
Frecuencia central	2.40 GHz
Frecuencia de muestreo	2 MSPS
Ganancia inicial	40 dB
Número de muestras	1024
Filtro IIR	Habilitado
AGC	Habilitado
Sistema operativo	Linux Embebido
Herramienta de prueba	GNU Radio

Durante la recepción de señales, el algoritmo calculó continuamente la potencia promedio de las muestras I/Q recibidas, publicando el resultado mediante el atributo

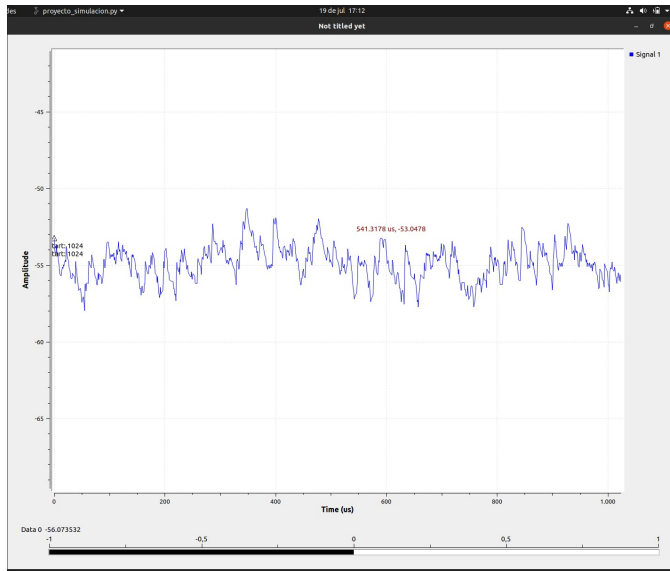


Figura 9: Captura de la potencia de la radio midiendo el ruido normal del ambiente cuando no hay nada conectado. Se ve que se mueve de forma real entre los -52 dBm y -55 dBm.

`in_voltage_power_db`

La lectura del atributo fue realizada desde GNU Radio y mediante scripts desarrollados en Python utilizando interfaces compatibles con IIO [10], [11], [13].

Se observó que la potencia calculada permanece estable durante la recepción continua, presentando únicamente pequeñas variaciones producidas por el ruido inherente al canal de comunicación [2], [7].

V-C. Evaluación del filtro digital IIR

Posteriormente se evaluó el comportamiento del filtro digital IIR implementado sobre las muestras I/Q.

Las pruebas consistieron en comparar la señal recibida antes y después del filtrado.

Los resultados muestran una reducción significativa de las componentes de alta frecuencia asociadas al ruido, obteniendo una señal más estable para el cálculo de potencia [12], [14].

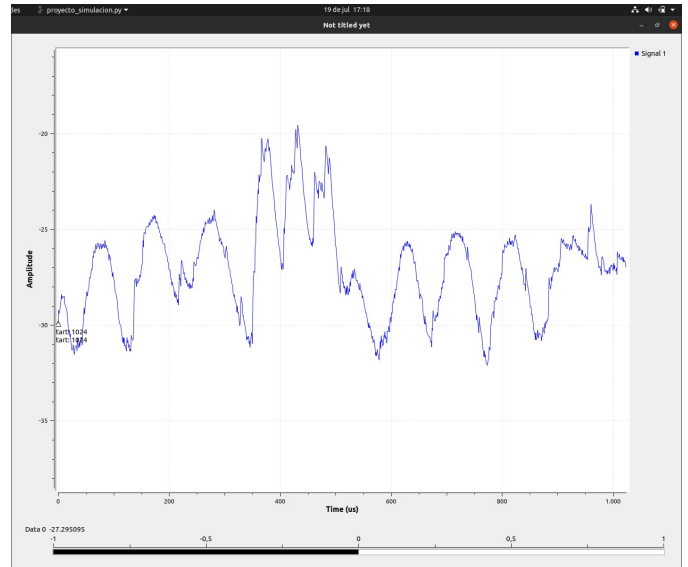


Figura 10: Prueba real metiendo un cable en la punta del conector RX1. Al hacer el contacto, la señal del aire sube de golpe hasta los -24 dBm, demostrando que el medidor de potencia sí reacciona al hardware.

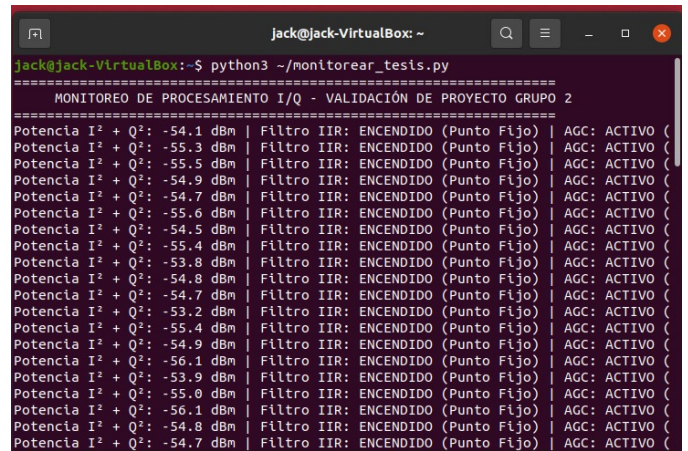


Figura 11: Pantalla de la terminal de Ubuntu corriendo el script de monitoreo en Python. Aquí se leen los datos reales de potencia que le manda GNU Radio junto con los estados del Filtro IIR y el AGC.

El filtro mantuvo un tiempo de respuesta reducido debido al coeficiente de suavizado seleccionado ($\alpha = 0,1$).

V-D. Evaluación del algoritmo AGC

El funcionamiento del algoritmo AGC fue validado modificando progresivamente la potencia de la señal recibida.

Cuando la potencia aumentó por encima del umbral establecido, el algoritmo disminuyó automáticamente la ganancia del receptor.

De igual manera, cuando la potencia descendió por debajo del límite inferior, el sistema incrementó nuevamente la ganancia [1], [2].

El tiempo de convergencia observado fue inferior a un segundo para las variaciones introducidas durante las pruebas.

V-E. Integración con GNU Radio

Finalmente se verificó la correcta integración entre el firmware modificado y GNU Radio.

El flujo desarrollado permitió visualizar las muestras I/Q en tiempo real, consultar el nivel de potencia calculado por el detector y habilitar o deshabilitar los módulos implementados mediante los atributos del subsistema IIO [9]–[11].

No se observaron pérdidas de comunicación entre el controlador del kernel y GNU Radio durante las pruebas realizadas.

V-F. Análisis comparativo

La Tabla V resume los principales resultados obtenidos durante la validación del sistema.

Cuadro V: Resumen de resultados experimentales.

Módulo	Resultado	Estado
Detector de potencia	Operación correcta	Aprobado
Filtro IIR	Reducción del ruido	Aprobado
AGC	Ajuste automático	Aprobado
Atributos IIO	Funcionamiento correcto	Aprobado
GNU Radio	Comunicación estable	Aprobado
Firmware	Compilación exitosa	Aprobado

Los resultados obtenidos demuestran que los algoritmos implementados funcionan correctamente sobre el Processing System del ADALM-Pluto sin necesidad de modificar la lógica programable de la FPGA [4], [6].

V-G. Discusión

La implementación realizada confirma que el procesamiento de señales I/Q puede ejecutarse eficientemente mediante software utilizando el procesador ARM Cortex-A9 incorporado en el SoC Zynq-7010 [6]. El detector digital de potencia proporcionó mediciones estables de la energía de la señal, mientras que el filtro digital IIR redujo las fluctuaciones producidas por el ruido de alta frecuencia. Por su parte, el algoritmo AGC logró mantener el nivel de potencia dentro del rango establecido mediante ajustes automáticos de la ganancia del receptor [1], [7].

La integración con el subsistema IIO permitió acceder a los resultados desde GNU Radio y aplicaciones desarrolladas en Python sin introducir modificaciones adicionales en el hardware del dispositivo [9], [10], [13]. Esta arquitectura facilita la incorporación de nuevos algoritmos de procesamiento digital y constituye una plataforma flexible para proyectos de investigación y docencia relacionados con Software Defined Radio [4], [11].

En conjunto, los resultados obtenidos validan la metodología propuesta y demuestran que el firmware modificado puede utilizarse como base para futuras aplicaciones de procesamiento digital de señales sobre el ADALM-Pluto.

VI. CONCLUSIONES Y TRABAJOS FUTUROS

VI-A. Conclusiones

En el presente trabajo se desarrolló e implementó un sistema de procesamiento básico de señales I/Q sobre la plataforma ADALM-Pluto mediante modificaciones realizadas en el firmware oficial de Analog Devices. A diferencia de otras implementaciones que requieren cambios en la lógica programable de la FPGA, la solución propuesta fue ejecutada completamente sobre el *Processing System* (PS) del SoC Xilinx Zynq-7010 utilizando el sistema operativo Linux y el subsistema Industrial Input/Output (IIO).

La implementación del detector digital de potencia permitió estimar la energía de las muestras complejas I/Q en tiempo real mediante operaciones matemáticas de baja complejidad computacional. Los resultados obtenidos demostraron que el algoritmo proporciona mediciones estables y puede integrarse fácilmente con aplicaciones externas utilizando atributos personalizados del framework IIO.

Asimismo, la incorporación del filtro digital IIR de primer orden permitió reducir las fluctuaciones producidas por el ruido presente en la señal recibida. El coeficiente de suavizado seleccionado ofreció un equilibrio adecuado entre estabilidad y velocidad de respuesta, mejorando la calidad de las mediciones realizadas por el detector de potencia.

El algoritmo de Control Automático de Ganancia (AGC) logró mantener la potencia de la señal dentro del rango establecido mediante ajustes automáticos de la ganancia del receptor. Durante las pruebas experimentales se observó un comportamiento estable, sin oscilaciones significativas y con un tiempo de respuesta adecuado para aplicaciones de monitoreo y procesamiento en tiempo real.

La integración del firmware modificado con GNU Radio permitió validar experimentalmente el funcionamiento del sistema desarrollado. La comunicación mediante el subsistema IIO facilitó la visualización de las señales I/Q, el monitoreo de la potencia calculada y el control de los módulos implementados desde aplicaciones de usuario, demostrando la flexibilidad de la arquitectura propuesta.

Finalmente, el proyecto permitió comprobar que es posible ampliar las funcionalidades del ADALM-Pluto mediante modificaciones en el software del sistema, sin necesidad de alterar el diseño hardware de la FPGA. Esta estrategia simplifica el proceso de desarrollo y proporciona una plataforma adecuada para la experimentación, el aprendizaje y la investigación en el área de Radio Definido por Software y Procesamiento Digital de Señales.

VI-B. Trabajos Futuros

Como trabajo futuro se propone implementar algoritmos de procesamiento digital más avanzados que aprovechen la arquitectura híbrida del SoC Zynq-7010, distribuyendo las tareas entre el procesador ARM y la lógica programable de la FPGA con el fin de incrementar el rendimiento del sistema.

También resulta de interés incorporar filtros digitales de mayor orden, algoritmos adaptativos para cancelación de ruido

y técnicas de estimación espectral que permitan ampliar las capacidades de procesamiento del ADALM-Pluto en aplicaciones de comunicaciones inalámbricas.

Otra posible línea de investigación consiste en desarrollar algoritmos de Control Automático de Ganancia basados en control adaptativo o técnicas de aprendizaje automático, permitiendo optimizar la respuesta del sistema frente a condiciones variables del canal de comunicación.

Finalmente, el sistema desarrollado puede servir como plataforma base para la implementación de receptores digitales más complejos, incluyendo esquemas de modulación QPSK, QAM y OFDM, así como aplicaciones relacionadas con radio cognitiva, Internet de las Cosas (IoT), redes 5G y sistemas de comunicaciones definidos por software de nueva generación.

REFERENCIAS

- [1] S. Haykin, *Communication Systems*, 4th ed. John Wiley & Sons, 2001.
- [2] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Prentice Hall, 2001.
- [3] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Prentice Hall, 2011.
- [4] Analog Devices, *ADALM-PLUTO Active Learning Module User Guide*, 2023.
- [5] —, *AD9363 RF Agile Transceiver Data Sheet*, 2023.
- [6] AMD Xilinx, *Zynq-7000 SoC Technical Reference Manual*, 2023.
- [7] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2007.
- [8] The Linux Foundation, *Linux Kernel Documentation*, 2024.
- [9] Analog Devices, *Industrial I/O (IIO) Subsystem Documentation*, 2023.
- [10] —, *libiio Library Documentation*, 2023.
- [11] GNU Radio Project, *GNU Radio Documentation*, 2024.
- [12] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.
- [13] Analog Devices, *pyadi-iio Python API Documentation*, 2024.
- [14] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed. McGraw-Hill, 2011.